LAPORAN DevSecOps
KELOMPOK 3 - Lab 3 VM - Ubuntu 24.04.3 LTS

Nama	: Muhammad Rafi Putra Hidayat
	Hafidz Kurnia Darojad
	Indra Rahmat Maulidi
NRP	: 3124600009
	3124600015
	3124600017
Dosen Pengajar	: Ferry Astika Saputra, ST., M.Sc

PROGRAM STUDI D4 TEKNIK INFORMATIKA

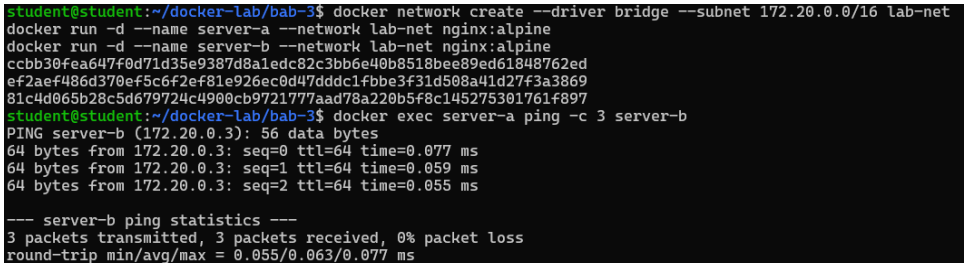
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)

TAHUN 2026

User-defined bridge network

Jalankan langkah berikut secara berurutan, lalu verifikasi hasil sebelum melanjutkan ke sub-langkah berikutnya.

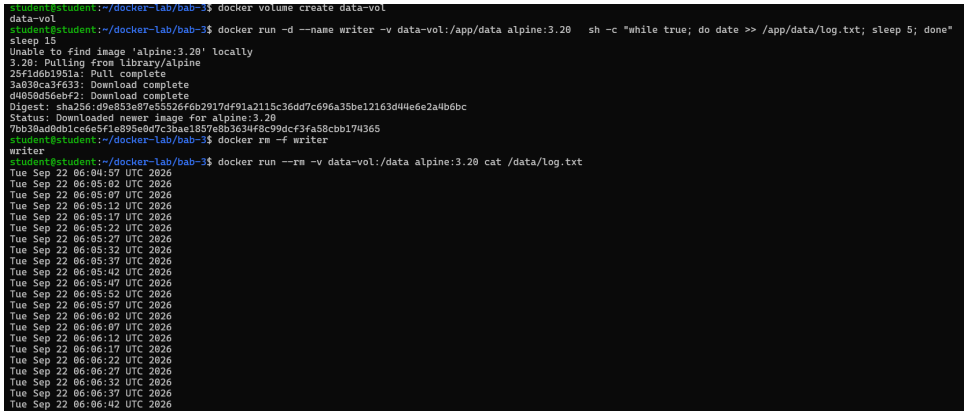
```
docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
docker run -d --name server-a --network lab-net nginx:alpine
docker run -d --name server-b --network lab-net nginx:alpine
docker exec server-a ping -c 3 server-b
docker rm -f server-a server-b
```



Volume backup dan restore

Jalankan langkah berikut secara berurutan, lalu verifikasi hasil sebelum melanjutkan ke sub-langkah berikutnya.

```
docker volume create data-vol
docker run -d --name writer -v data-vol:/app/data alpine:3.20   sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
sleep 15
docker rm -f writer
docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
```




```
docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20   tar czf /backup/data-vol-backup.tar.gz -C /source
```



compose multi-container Nginx-Flask-PostgreSQL

Arsitektur Laboratorium dan Prasyarat Lingkungan

```
mkdir -p ~/docker-lab/bab-3/app
mkdir -p ~/docker-lab/bab-3/html
cd ~/docker-lab/bab-3
```

Struktur akhirnya

```
bab-3/
├── compose.yaml
├── nginx.conf
├── html/
│   └── index.html
└── app/
    ├── Dockerfile
    ├── requirements.txt
    └── app.py
```

Isi:

```
GNU nano 7.2                                                    compose.yaml
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - frontend
    depends_on:
      - app

  app:
    build:
      context: ./app
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks:
      - frontend
      - backend
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks:
      - backend
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U labuser -d labdb
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pg-data:

networks:
  frontend:
  backend:
```

Buat app/requirements.txt

```
  GNU nano 7.2
Flask==3.1.2
psycopg[binary]==3.2.9
gunicorn==23.0.0
```

Buat app/app.py:

```
 GNU nano 7.2                                                                app/app.py
import os

import psycopg
from flask import Flask, jsonify

app = Flask(__name__)


def database_connection():
    return psycopg.connect(
        host=os.environ["DB_HOST"],
        port=os.getenv("DB_PORT", "5432"),
        dbname=os.environ["DB_NAME"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASS"],
    )


@app.get("/")
def index():
    try:
        with database_connection() as connection:
            with connection.cursor() as cursor:
                cursor.execute("SELECT version();")
                database_version = cursor.fetchone()[0]

        return jsonify(
            status="ok",
            message="Nginx, Flask, dan PostgreSQL berhasil terhubung",
            database=database_version,
        )
    except Exception as error:
        return jsonify(
            status="error",
            message="Koneksi database gagal",
            detail=str(error),
        ), 500


@app.get("/health")
def health():
    try:
        with database_connection() as connection:
            with connection.cursor() as cursor:
                cursor.execute("SELECT 1;")
                cursor.fetchone()

        return jsonify(status="healthy"), 200
    except Exception:
        return jsonify(status="unhealthy"), 503
```

Buat app/Dockerfile:

```
 GNU nano 7.2                                                             app/Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN useradd --system --uid 10001 appuser
USER appuser

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

Buat nginx.conf:

```
 GNU nano 7.2                                                                nginx.conf
upstream flask_backend {
    server app:5000;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://flask_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }

    location = /static.html {
        root /usr/share/nginx/html;
    }
}
```

Buat halaman statis index.html:

```
  GNU nano 7.2                                                                  html/index.html
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Laboratorium Docker Compose</title>
</head>
<body>
    <h1>Docker Compose Bab 3</h1>
    <p>Bind mount Nginx berhasil digunakan.</p>
</body>
</html>
```

Menjalankan Stack

```
student@student:~/docker-lab/bab-3$ docker compose up -d --build
[+] up 13/13
 ✔Image postgres:16-alpine Pulled                                                31.0s
[+] Building 26.4s (13/13) FINISHED
 => [internal] load local bake definitions                                       0.0s
 => => reading from stdin 504B                                                   0.0s
 => [internal] load build definition from Dockerfile                             0.1s
 => => transferring dockerfile: 307B                                             0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim              2.9s
 => [internal] load .dockerignore                                                0.1s
 => => transferring context: 2B                                                  0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a83de48e70534b94cd  10.9s
 => => resolve docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a83de48e70534b94cd8  0.1s
 => => sha256:06ad939ed42b51caafb25b14810875d14e0ee0c4d1d3e2e5e4d475fb8c150bca 249B / 249B                       0.3s
 => => sha256:b0dc7f87bef15c2536cb182d6c48b52f90a852288ec9eb222f833fa97ffebf0 12.12MB / 12.12MB                  3.3s
 => => sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab9945550 1.29MB / 1.29MB                       2.7s
 => => sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db8d8115157024c8 29.83MB / 29.83MB                 8.5s
 => => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db8d8115157024c8                        1.0s
 => => extracting sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab9945550 0a76                       0.2s
 => => extracting sha256:b0dc7f87bef15c2536cb182d6c48b52f90a852288ec9eb222f833fa97ffebf0                        0.6s
 => => extracting sha256:06ad939ed42b51caafb25b14810875d14e0ee0c4d1d3e2e5e4d475fb8c150bca                       0.1s
 => [internal] load build context                                                0.3s
 => => transferring context: 1.40kB                                              0.0s
 => [2/6] WORKDIR /app                                                           1.0s
 => [3/6] COPY requirements.txt .                                                0.2s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                     6.8s
 => [5/6] COPY app.py .                                                          0.2s
 => [6/6] RUN useradd --system --uid 10001 appuser                               0.6s
 => exporting to image                                                           2.9s
 => => exporting layers                                                          1.9s
 => => exporting manifest sha256:232c84958bab84a14f6ceab0cc49fa1e498b1644c4f45615069ffb522ec420f3                0.0s
 => => exporting config sha256:8b97cfc2f4292e74fbd8192d2c89b97543c2df971fce7fbbfa04473163f8bf25f                 0.0s
 => => exporting attestation manifest sha256:81d5db5f828067420853e8d8aa547dcbc58c11701348aeff31c86b309b8487c9    0.1s
 => => exporting manifest list sha256:14e9362eb3bf3efdae6d57fcc6da1e6437a0c50785df851b8c434f5daba255              0.0s
 => => naming to docker.io/library/bab-3-app:latest                              0.0s
[+] up 20/20ring to docker.io/library/bab-3-app:latest                           0.7s
 ✔Image postgres:16-alpine Pulled                                                31.0s
 ✔Image bab-3-app          Built                                                 26.5s
 ✔Network bab-3_backend    Created                                                0.2s
 ✔Volume bab-3_pg-data     Created                                                0.0s
 ✔Network bab-3_frontend   Created                                                0.1s
 ✔Container bab-3-db-1     Healthy                                                6.6s
 ✔Container bab-3-app-1    Started                                                6.5s
 ✔Container bab-3-web-1    Started                                                6.8s
student@student:~/docker-lab/bab-3$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED              STATUS                        PORTS                          NAMES
2c98d892f21c   nginx:alpine       "/docker-entrypoint.…"   About a minute ago   Up About a minute             127.0.0.1:8080->80/tcp         bab-3-web-1
f179e97e82e7   bab-3-app          "gunicorn --bind 0.0…"   About a minute ago   Up About a minute             5000/tcp                       bab-3-app-1
d4afefb4d930   postgres:16-alpine "docker-entrypoint.s…"   About a minute ago   Up About a minute (healthy)   5432/tcp                       bab-3-db-1
```

Verifikasi dan Skenario Pengujian

[✅] Container di user-defined bridge dapat saling resolve menggunakan nama.

```
student@student:~/docker-lab/bab-3$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
docker run -d --name server-a --network lab-net nginx:alpine
docker run -d --name server-b --network lab-net nginx:alpine
ccbb30fea647f0d71d35e9387d8a1edc82c3bb6e40b8518bee89ed61848762ed
ef2aef486d370ef5c6f2ef81e926ec0d47dddc1fbbe3f31d508a41d27f3a3869
81c4d065b28c5d679724c4900cb9721777aad78a220b5f8c145275301761f897
student@student:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.077 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.059 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.055 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.055/0.063/0.077 ms
```

[✅] Data di named volume tetap ada setelah container dihapus.

```
student@student:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Tue Sep 22 06:23:28 UTC 2026
Tue Sep 22 06:23:33 UTC 2026
Tue Sep 22 06:23:38 UTC 2026
Tue Sep 22 06:23:43 UTC 2026
Tue Sep 22 06:23:48 UTC 2026
Tue Sep 22 06:23:53 UTC 2026
Tue Sep 22 06:23:58 UTC 2026
```

[✅] Bind mount menunjukkan perubahan file host tanpa rebuild image.

```
student@student:~/docker-lab/bab-3$ curl http://localhost:8080/static.html
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Laboratorium Docker Compose</title>
</head>
<body>
    <h1>Docker Compose Bab 3</h1>
    <p>Bind mount Nginx berhasil digunakan.</p>
</body>
</html>
student@student:~/docker-lab/bab-3$ echo "<h1>Versi Kedua - Berhasil Diubah Tanpa Rebuild</h1>" > html/static.html
student@student:~/docker-lab/bab-3$ curl http://localhost:8080/static.html
<h1>Versi Kedua - Berhasil Diubah Tanpa Rebuild</h1>
```

[✅] tmpfs kehilangan data setelah container restart.

```
student@student:~/docker-lab/bab-3$ docker run -d --name test-tmpfs --tmpfs /app/cache alpine:3.20 sleep 3600
9524459fb9c2527639d4858e0fb1c5d574e08a97531cb3c74e2da8c517c2adf8
student@student:~/docker-lab/bab-3$ docker exec test-tmpfs sh -c "echo 'rahasia-sementara-123' > /app/cache/token.txt"
student@student:~/docker-lab/bab-3$ docker exec test-tmpfs cat /app/cache/token.txt
rahasia-sementara-123
student@student:~/docker-lab/bab-3$ docker restart test-tmpfs
test-tmpfs
student@student:~/docker-lab/bab-3$ docker exec test-tmpfs cat /app/cache/token.txt
cat: can't open '/app/cache/token.txt': No such file or directory
```

[✅] Compose stack web-app-db berjalan dan API health menampilkan koneksi database.

```
student@student:~/docker-lab/bab-3$ docker compose ps
NAME          IMAGE              COMMAND                  SERVICE   CREATED          STATUS                    PORTS
bab-3-app-1   bab-3-app          "gunicorn --bind 0.0…"   app       10 seconds ago   Up 3 seconds              5000/tcp
bab-3-db-1    postgres:16-alpine "docker-entrypoint.s…"   db        10 seconds ago   Up 9 seconds (healthy)    5432/tcp
bab-3-web-1   nginx:alpine       "/docker-entrypoint.…"   web       10 seconds ago   Up 3 seconds              127.0.0.1:8080->80/tcp
student@student:~/docker-lab/bab-3$ curl -i http://localhost:8080/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Tue, 22 Sep 2026 06:48:31 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive

{"status":"healthy"}
```

Evaluasi dan Latihan Mandiri

1. Mengapa user-defined bridge lebih baik daripada default bridge untuk multi-container app?

User-defined bridge menyediakan resolusi DNS otomatis berdasarkan nama service. Container dapat saling berkomunikasi menggunakan nama tanpa perlu mengetahui alamat IP dinamis yang bisa berubah.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount memberikan akses langsung ke sistem file host. Jika tidak dibatasi dengan flag read-only (:ro), proses di dalam container (terutama jika berjalan sebagai root) memiliki hak untuk memodifikasi, menimpa, atau menghapus file di host. Ini membuka celah keamanan yang serius, di mana penyerang yang berhasil masuk ke container dapat melakukan privilege escalation.

3. Apa perbedaan docker compose down dan docker compose down -v?

Docker compose down mematikan dan menghapus container dan network yang dibuat dalam project tapi tetap menyimpan volume. Flag -v menyebabkan volume juga ikut terhapus.

4. Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?

Saat butuh database untuk online dan jalan dengan normal terlebih dahulu (biasanya db butuh waktu untuk siap).

5. Bagaimana strategi backup volume untuk database produksi?

Menjalankan container sementara yang me-mount volume database tersebut, lalu mengeksekusi perintah dump native (misal pg_dump) untuk menghasilkan file backup yang konsisten secara logika. File backup tersebut kemudian harus dienkripsi dan dikirim ke penyimpanan eksternal yang terpisah (misal cloud S3).